Decision Making and Branching

Introduction:

We know that 'C' program is set of statements or instructions which are normally executed in sequence order. In fact to execute the instructions in sequence manner, we don't have to do anything. That is by default the instructions in 'C' language are executed in sequence manner.

However, there are number of situations where we have to change the order of execution based on certain condition, for that purpose 'C' language provides some decision making statements and they are as follows:

1) Conditional operator (Ternary operator) statement
2) If statement
3) Switch statement
4) goto statement.

All above decision making statements are also called as 'Control Statements' because they controls the execution flow of program.

Let's see all these control statements in details:

1) Conditional Operator Statement:

Conditional operator statement is also called as ternary operator statement. There are two ternary or conditional operators in 'C' language which are ? and :

The working of conditional operator statement is given in following syntax:

Syntax:

```
Exp1 ? Exp2 : Exp3 ;
```

Here,

The *Exp1* is evaluated first, if it is True then value of *Exp2* is calculated and value of *Exp2* becomes value of entire expression.

If the value of *Exp1* is False then value of *Exp3* is calculated and value of *Exp3* becomes value of entire expression.

e.g

```
//Program to check entered number is positive or negative.
#include< stdio.h >
#include< conio.h >
void    main( )
{
        int     a ;
        clrscr( );
        printf( " Enter any number ");
        scanf("%d", &a);
        ( a > 0 ) ? printf(" Number is + ve") : printf("Number is – ve" );
        getch( );
}
```

OUTPUT:

Enter any number -7

Number is – ve

2) if Statement: (Two way decision making statement)

if statement is more powerful decision making statement and used to control execution flow of program. If statement is useful to check particular condition and take certain decision.

The general form of if statement is follows:

```
if ( condition )
{
        Block of statement ;
}
```

In above form, the Block of statement is executed if and only if condition is true, if condition false then Block of statement is not executed.

The if statement having different types or forms depending upon condition and which are as follows:

1) Simple if statement 2) if-else statement

3) Nested if-else statement 4) else-if ladder

1) Simple if statement:

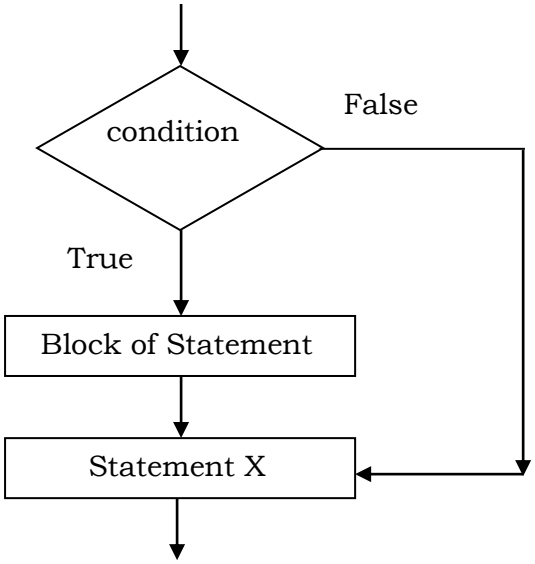
The general form of simple if statement is as follows:

```
if ( condition )
{
        Block of statement;
}
Statement X;
```

In above general form, Block of statement may be single statement or group of statements.

If the condition is true then Block of statement is executed otherwise the Block of statement will skipped and control jumps towards *StatementX*

The logic of simple if statement is shown in following fig:



e.g.

```
int      z = 10 ;

if ( z = = 10)
{
    printf(" Number is Ten");
}
```

OUTPUT:

Number is Ten

2) if-else statement:

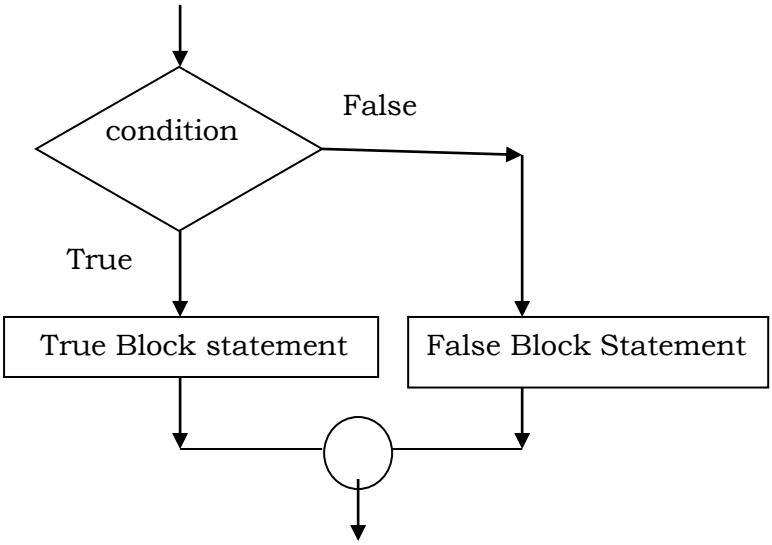
The general form of if-else statement is as follows:

```
if ( condition )
{
    True Block statement;
}
else
{
    False Block Statement;

}
```

In above general form, True Block statement is executed if and only if the condition is true. If condition False then False Block Statement is executes.

The logic of if-else statement is shown in following fig:



e.g.

Program to check maximum number between two numbers:

```
#include< stdio.h >
#include< conio.h >
void      main( )
{
        int        a,b;
        clrscr( );
        printf( " Enter any two numbers ");
        scanf("%d%d",&a,&b);
        if ( a > b )
        {
            printf( " %d is maximum ", a );
        }
        else
        {
            printf( " %d is maximum ", b );
        }
        getch( ) ;
}
```

<u>OUTPUT:</u> Enter any two number 5 45

45 is maximum

3) Nested if-else statement:

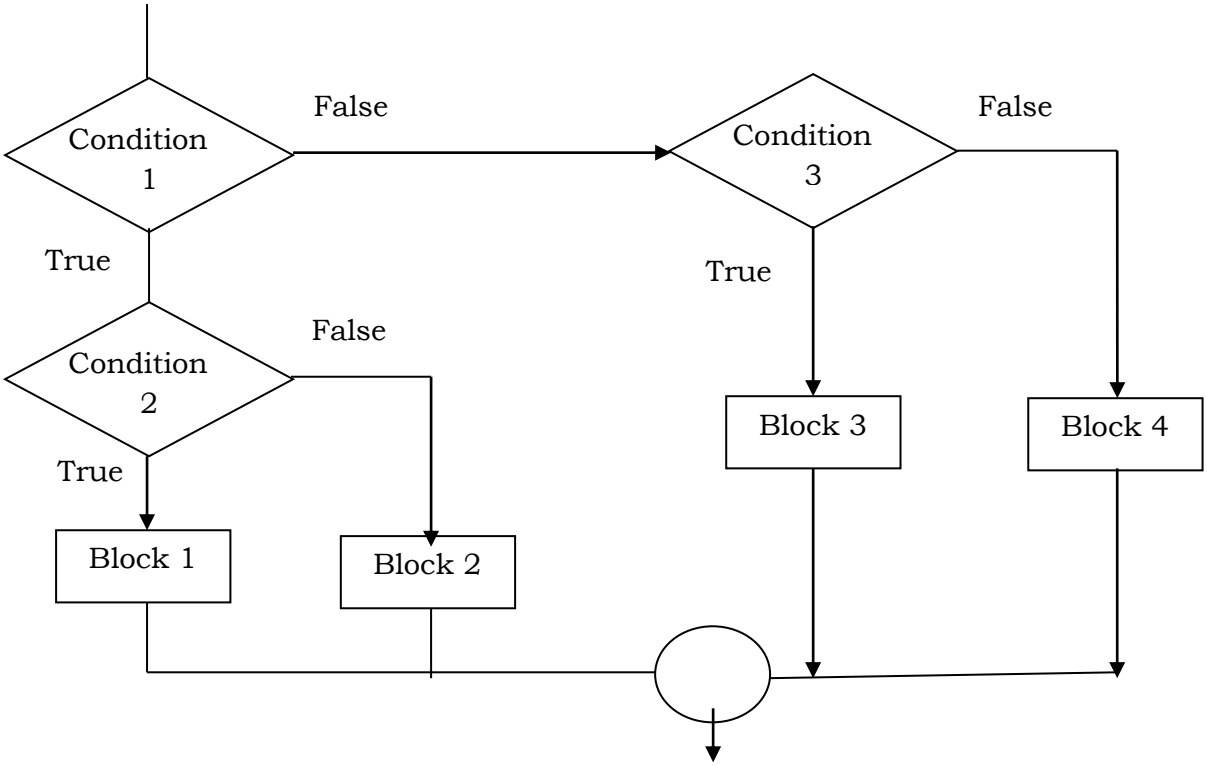
When a series of decisions are involved at that time we may have to use more than one if-else statement in nested form as follows:

```
if ( condition 1)
{
        if ( condition 2)
        {
            Block 1;
        }
        else
        {
            Block 2;
        }
 }
else
{
        if ( condition 3)
        {
            Block 3;
        }
        else
        {
            Block 4;
        }
 }
```

In above nested if-else statement, first *condition1* is checked if it is true then control enters in the body of outer if statement and *condition 2* is checked, if *condition 2* is true then *Block 1* is executed if *condition 2* is false then *Block 2* is executed.

If *condition 1* is false then control goes towards else and the body of outer else statement is executed in that *condition 3* is checked, if *condition 3* is true then *Block 3* is executed if *condition 3* is false then *Block 4* is executed.

The logic of nested if-else statement is shown in following fig:



e.g. program to check entered number is zero or positive or negative

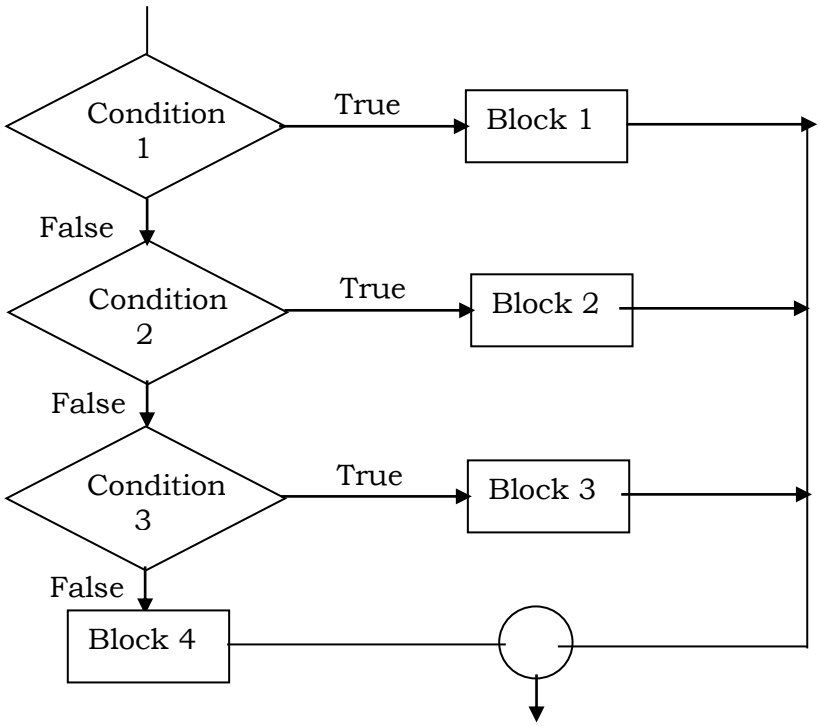
```
#include<stdio.h>
#include<conio.h>
void   main( )
{
        int     n;
        clrscr( );
        printf("Enter any number");
        scanf("%d",&n);
        if ( n > = 0)
        {
                if ( n = = 0)
                {
                        printf("Number is Zero");
                }
               else
                {
                        printf("Number is Positive");
                }
        }
        else
        {
        printf("Number is Negative");
        }
getch( );
}
```

4) <u>else-if ladder:</u>

There is another way of putting if's together when more than one condition we have to check. In else-if ladder, each if statement having their own else statement. The general form of else-if ladder is as follows:

```
if ( condition 1)
{
        Block 1;
 }
else    if( condition 2)
{
        Block 2;
 }
else    if( condition 3)
{
        Block 3;
}
else
{
        Block 4;
}
```

The logic of above else-if ladder is shown as following fig:



e.g. Program to find minimum number between three numbers

```
#include<stdio.h>
#include<conio.h>
void   main( )
{
        int     a,b,c;
        clrscr( );
        printf("Enter any Three numbers");
        scanf("%d%d%d",&a,&b,&c);
        if ( a<b && a<c )
        {
                printf("%d is minimum", a );
        }
        else   if (  b<a && b<c )
        {
                printf("%d is minimum", b );
        }
         else
        {
                printf("%d is minimum", c );
        }
getch( );
}
```

OUTPUT: Enter any Three numbers 7 8 3
3 is minimum

3) switch statement: (Multi way decision making statement)

The control statement that allows us to make a decision from number of choices is called "switch statement" or "switch-case-default" statement.

Syntax:

```
switch( expression )
{
      case    value1:
              Block1;
              break;
      case    value2:
              Block2;
              break;
                ⋮
                ▼
      case    valueN:
              BlockN;
              break;
      default:
              Default Block;
}
```

In given syntax of switch statement:

- expression is any valid expression or variable
- value1, value2,. . . . . ,valueN are case values or case labels and they must be unique and constant. And end of each case labels is colon(:)
- switch, case, default and break are the keywords

➢ Working of switch statement:

The switch statement checks value of expression or variable with first case value, if value matches then *Block1* is executed.

If value of expression does not match with first case value then switch statement check value of expression with case value2. If expression value matches with case value2 then *Block2* is executed.

This process of comparison is happen until successful choice is made.

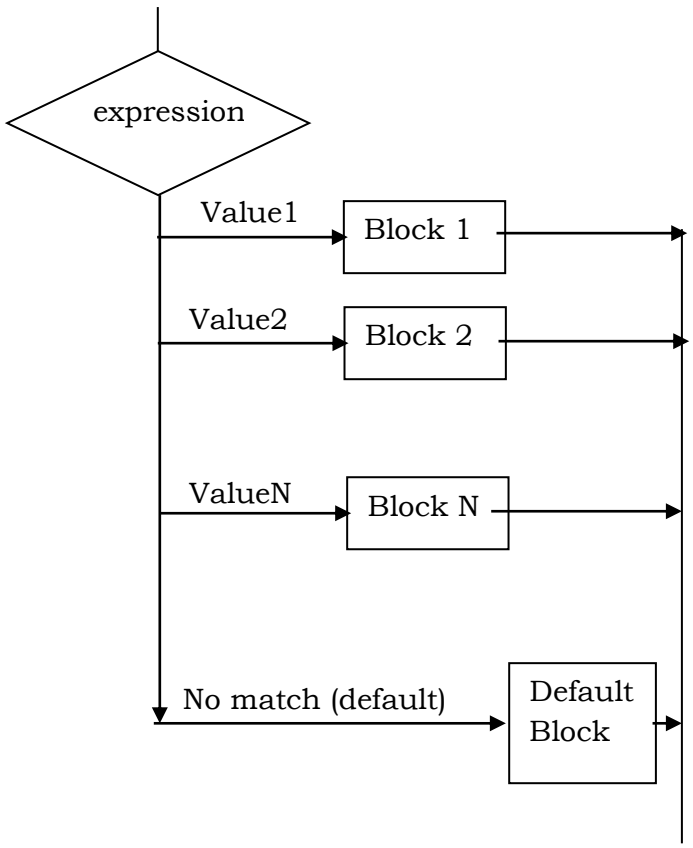
If value of expression does not match with any case value then default block is executed.

NOTE:

The break statement in each case block causes immediate exit from the switch statement.

Generally, break statement is used to exit from any control statement.

The logic of switch statement is as follows:



e.g. Program which accept single digit & prints corresponding week day.

```
#include<stdio.h>
#include<conio.h>
void   main( )
{
        int     n;
        clrscr( );
        printf("Enter any single digit number");
        scanf("%d",&n);
        switch( n )
        {
                case   1:
                        printf("Sunday");
                        break;
                case   2:
                        printf("Monday");
                        break;
                case   3:
                        printf("Tuesday");
                        break;
                case   4:
                        printf("Wednesday");
                        break;
                case   5:
                        printf("Thursday");
                        break;
                case   6:
                        printf("Friday");
                        break;
                case   7:
                        printf("Saturday");
                        break;
                default:
                        printf("Wrong Input");
        }
getch( );
}
```

<u>OUTPUT:</u> Enter any single digit number 56
Wrong Input

4) goto statement:

The 'C' language supports the goto statement to jump(branch) unconditionally from one place to another place in the program.

Syntax:

```
goto    label;
                statement1;
                statement2;
                statementN;

        label :
                statementA;
                staementB;
                statementN;
```

In above syntax, The goto statement requires label(any identifier) to identify the place at which control jumps in short label is place at which control jumps.

Depending upon the place of label definition, the goto statement having two types:

1) Forward jump:

"The goto statement in which label definition is after the goto statement such a jump is called as forward jump"

Following syntax shows the forward jump:

```
goto    label;
                statement1;
                statement2;
                statementN;

        label :
                statementA;
                staementB;
                statementN;
```

In forward jump the label definition is placed after goto statement therefore some statements are not executed such statements are called as 'skipped statements'

e.g. Following example shows the use of forward jump.

```
#include<stdio.h>
#include<conio.h>
void   main( )
{
        clrscr( );
        printf("\n SUNDAY");
        goto    day;
                printf("\n MONDAY");
                printf("\n TUESDAY");
                printf("\n WEDNESDAY");
        day:
                printf("\n FRIDAY");
                printf("\n SATURDAY");
getch( );
}
```

OUTPUT:

```
SUNDAY
FRIDAY
SATURDAY
```

2) Backward jump:

"The goto statement in which label definition is before the goto statement such a jump is called as backward jump"

Following syntax shows the backward jump:

```
label:
        statement1;
        statement2;
        statementN;
goto  label;
        statementA;
        staementB;
        statementN;
```

In case of back ward jump, label definition is before the goto statement therefore some statements are repeatedly executed i.e. back ward jump creates the loop.

e.g.

Program to print the series like 1 2 3 4 5 by using back ward jump.

```
#include<stdio.h>
#include<conio.h>
void   main( )
{
        int     n;
        clrscr( );
        n = 1 ;
        take:
                printf("\t %d", n);
                n + + ;
                if ( n <= 5)
                {
                        goto    take;
                }
        getch( );
}
```

<u>OUTPUT:</u>

1 2 3 4 5

Iteration logic (Looping)

Introduction:

We know that to develop any program there are three kinds of logics were used viz (namely) Sequence logic, Selection logic or Iteration logic.

In Iteration logic the block of statement is repeatedly executed till the condition is true and exit from the loop when condition becomes false. Every loop having two segments (parts) viz

1) Body of loop 2) Control statement (Condition)

The control statement tests the condition and executes body of loop repeatedly till the condition is true.

Depending upon the position of control statement in the loop, the loop can be classified into two categories

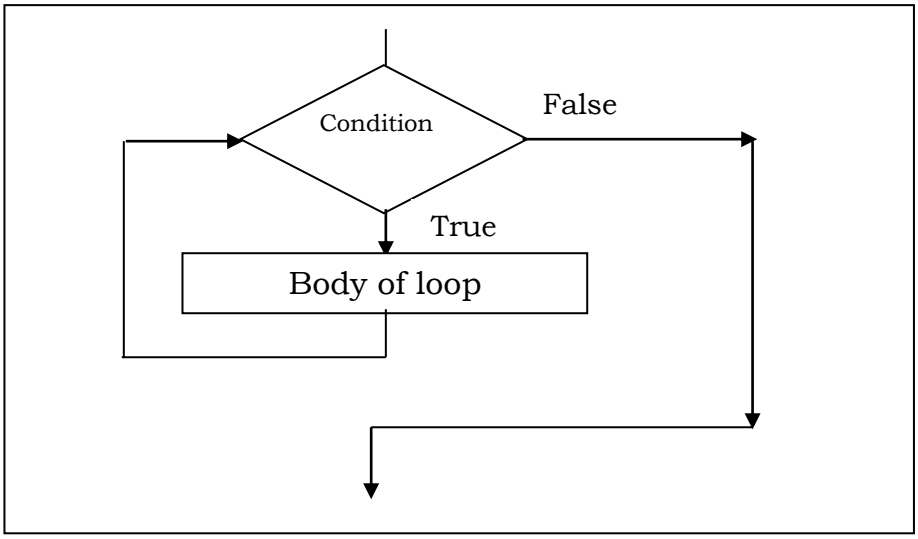
I) Entry Controlled loop II) Exit Controlled loop.

Let's see all these loops in details:

I) Entry Controlled loop:

In case of entry controlled loop, the counter condition is checked first if it is true then body of loop repeatedly executed till condition is true. If condition becomes false then control exit from body of loop.

The logic of entry controlled loop is shown in following fig:

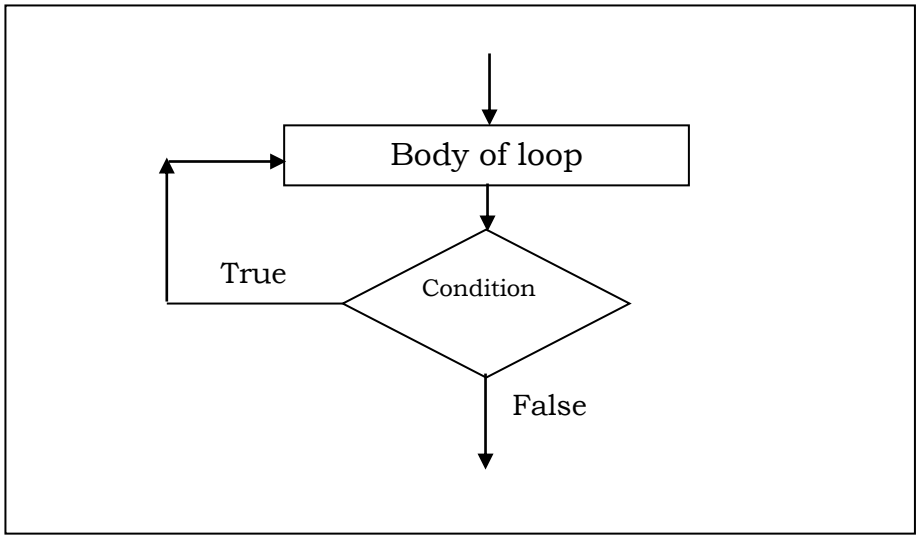


The *while loop* and *for loop* are the entry controlled loop.

II) Exit Controlled loop:

In case of exit controlled loop, first body of loop is executed and then counter condition is checked, if counter condition is true then body of loop is repeatedly executed till condition is true. If condition becomes false then control exit from body of loop.

The logic of entry controlled loop is shown in following fig:



The *do-while loop* is exit controlled loop.

Note:

- In case of exit controlled loop, if counter condition is false at first attempt still body of loop is executed at one time.
- In case of entry controlled loop, if counter condition is false at first attempt then body of loop never executed.

Also there are three important things behind every loop and they are:

I) Initialization of counter
II) Test counter condition
III) Increment / Decrement counter.

To perform well looping, we must have to use above three pointes at appropriate place.
The 'C' language has three kinds of looping statements viz
1) while loop 2) do-while loop 3) for loop
Lets see all these looping statements in details:

1) while loop:

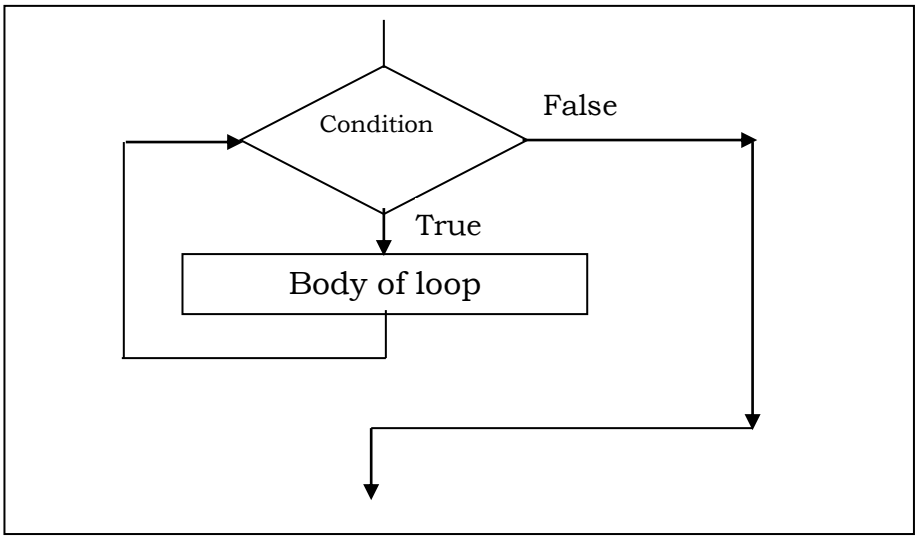
The while loop is entry controlled loop i.e. counter condition is checked first, if it is True then body of loop is executed repeatedly till condition is True. If condition becomes False then control exit from the body of loop.

while loop is simplest loop in all looping statements in 'C' language. The general form of while loop is as fallow:

```
while ( counter condition )
{
        Body of loop ;
}
```

In above general form, body of loop is executed repeatedly till condition is True. If condition becomes False then control exit from the body of loop.

Working of *while loop* is shown in following figure:



e.g.

Program to print 1 to 5 numbers by using while loop:

```
#include<stdio.h>
#include<conio.h>
void   main( )
{
        int     i;
        clrscr( );
        i = 1 ;      // initialization of counter

        while( i<=5 )    // test counter condition
        {
                printf("\t %d ", i );
                i ++;      // increment counter
        }
        getch( );
}
```

2) do-while loop:

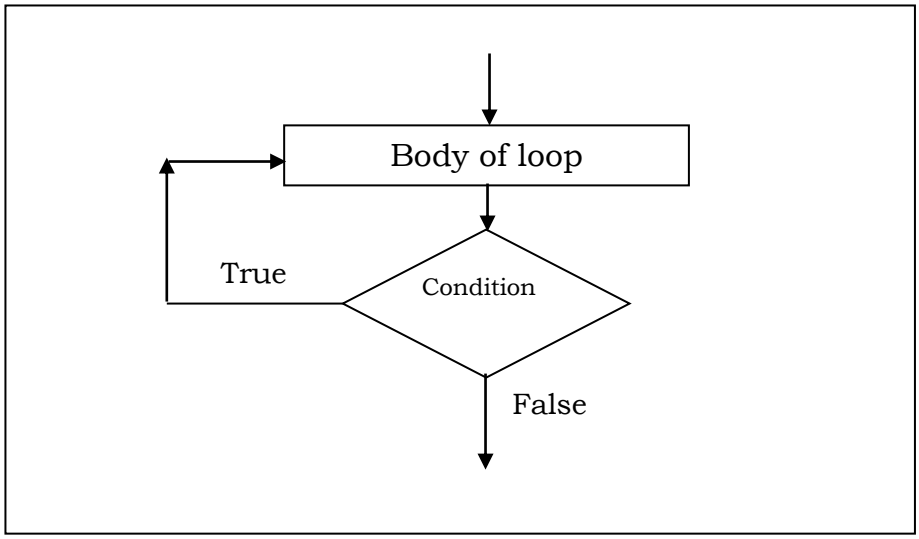
The do-while loop is exit controlled loop i.e. body of loop is executed first and then counter condition is checked. If condition is True then body of loop executed repeatedly till condition is True and exit from loop when condition becomes False.

Note that: At first attempt if counter condition is false still body of do-while loop executed at one time.

The general form of *do-while* loop is as fallows:

```
do
{
        Body of loop ;
} while (counter condition) ;
```

Working of do-while loop is shown in following figure:



e.g.

Program to print 1 to 5 numbers by using do-while loop:

```
#include<stdio.h>
#include<conio.h>
void   main( )
{
        int     i;
        clrscr( );
        i = 1 ;      // initialization of counter
        do
        {
                printf("\t %d ", i );
                i ++;      // increment counter
        } while( i<=5 ) ;    // test counter condition
        getch( );
}
```

3) for loop:

The for loop is also entry controlled loop i.e. counter condition is checked first, if it is True then body of loop is executed repeatedly till condition is True. If condition becomes False then control exit from the body of loop.

The general form of *for loop* is shown as fallow:

```
for ( initialization of counter ; test counter condition ; increment or decrement counter )
{
        Body of loop ;
}
```

The execution of *for loop* is done by following manner:

1) First initialization of counter is done by using assignment operator.
2) After initialization, control checks the counter condition if it is True then body of loop executed otherwise loop terminates (Exit or end)
3) After execution of body of loop, the control transferred back towards incrementation or decrementation.
   If incrementation or decrementation is done then control again checks counter condition if it is true then again body of loop is executed.
   This process is repeatedly done till condition is True and exit from loop if counter condition becomes false.

e.g.

Program to print 1 to 5 numbers by using *for loop*:

```
#include<stdio.h>
#include<conio.h>
void   main( )
{
        int     i;
        clrscr( );
        for ( i = 1 ; i <= 5 ; i ++ )
        {
                printf("\t %d ", i );
        }
        getch( );
}
```

******** ***Refer Notebook for Programs or Examples*** *********